

How would you implement the equivalent of /co-area-drivers in SQL? Describe the query structure (no need to write exact syntax).

There would be drivers, areas and trips tables (latter with foreign keys per trip to the first two). First, filter trips table to get dropoff areas of desired driver. Next, self-join trips table on dropoff areas to get pairs of drivers that shared an area. Then, get all the entries where the first driver is the desired driver and the second driver isn't. This should be grouped by the second driver, then count and order based on distinct areas.

Why is this traversal more natural and typically more efficient in a native graph database, referencing index-free adjacency?

Tracing a path where it exists via edges from drivers out to area nodes to other drivers is a very literal and hence natural representation for connectedness found in the query. With index-free adjacency, each node stores direct references to adjacent relationships and nodes, so this tracing can be done directly via the edges of the graph without needing the overhead stemming from expensive indexes and joins that relational databases need, especially as traversal gets complex.

Describe one workload characteristic of the taxi dataset that would cause Neo4j to underperform a relational database.

Any workload that is focused on reporting aggregation and analytics on the full data. For example, calculating global averages of a metric across the whole data set, the top drivers and companies, and other aggregated reporting that is what operations like GROUP BY were made for. Neo4j would struggle here as its graph-based layout requires more overhead to traverse all the pointers, has the relevant information spread-out across all nodes (bad locality) leading to more random disk access as opposed to one column and doesn't have the optimizations like vectorization and pipelines built in.